

Skills der Zukunft in der Softwarebranche

Kompetenzen und Qualifikationen im Zeitalter von KI und digitaler
Transformation

**Seminararbeit zur Vorlesung
Integrationsseminar**

des Studienganges Informatik
an der DHBW Ravensburg

von Max Mustermann

Kurs:	TINF23B
Matrikelnummer:	1234567
Dualer Partner:	SAP SE, 88677 Markdorf
Projektbetreuer:	Erika Musterfrau
Gutachter DHBW:	Prof. Dr. Hans Beispiel
Abgabedatum:	02.07.2026

Sperrvermerk

gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge der Dualen Hochschule Baden-Württemberg vom 29.09.2017:

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anderslautende Genehmigung des Dualen Partners vorliegt.

Ort, Datum

Unterschrift

Inhaltsverzeichnis

Abkürzungsverzeichnis.....	IV
Abbildungsverzeichnis.....	V
Tabellenverzeichnis.....	VI
1 Einleitung und Grundlagen.....	1
1.1 Motivation und Zielsetzung.....	1
1.2 Abgrenzung und Aufbau.....	1
1.3 Begriffliche Grundlagen.....	2
2 Technische Kernkompetenzen.....	3
2.1 KI-Kompetenz und Prompt Engineering.....	3
2.2 Cloud-native Entwicklung und DevOps.....	4
2.3 Systemverständnis und Architekturkenntnisse.....	4
3 Methodische und prozessuale Kompetenzen.....	6
3.1 Agile Entwicklungsmethoden und kontinuierliches Liefern.....	6
3.2 Qualitätssicherung und testgetriebene Entwicklung.....	6
3.3 Sicherheitskompetenz (Security by Design).....	7
4 Überfachliche Kompetenzen.....	8
4.1 Kommunikation und Kollaboration.....	8
4.2 Lernbereitschaft und Adaptionfähigkeit.....	8
4.3 Kritisches Denken und Problemlösekompetenz.....	9
5 Fazit und Ausblick.....	11
5.1 Zusammenfassung der Ergebnisse.....	11
5.2 Handlungsempfehlungen.....	12
5.3 Ausblick.....	12
Anhang.....	14
Literaturverzeichnis.....	15

Abkürzungsverzeichnis

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Delivery
DevOps	Development and Operations
IaC	Infrastructure as Code
KI	Künstliche Intelligenz
LLM	Large Language Model
MLOps	Machine Learning Operations
NLP	Natural Language Processing
SRE	Site Reliability Engineering
UX	User Experience

Abbildungsverzeichnis

Abbildung 1 Produktivitätssteigerung durch KI-Assistenten nach Aufgabentyp (vgl. (GitHub, Inc., 2023, S. 14 ff.)).....	3
Abbildung 2 Schematische Darstellung einer CI/CD-Pipeline (eigene Darstellung).	6
Abbildung 3 T-förmiges Kompetenzprofil als Karrieremodell (eigene Darstellung).	9

Tabellenverzeichnis

Tabelle 1	Kompetenzmodell für die Softwarebranche der Zukunft (eigene Darstellung). . .	11
Tabelle 2	Übersicht über die Nutzung von KI-basierten Werkzeugen.	14

1 Einleitung und Grundlagen

Die Softwarebranche befindet sich in einem tiefgreifenden und beschleunigten Wandel. Technologien wie Künstliche Intelligenz (KI), Cloud-Computing und automatisierte Entwicklungspipelines verändern nicht nur die Art, wie Software entsteht, sondern auch die Anforderungen an die Menschen, die sie entwickeln (vgl. (World Economic Forum, 2023, S. 4 f.)). Gleichzeitig steigt die gesellschaftliche Abhängigkeit von digitalen Systemen kontinuierlich: Kritische Infrastrukturen, Gesundheitsversorgung, Mobilität und Finanzwesen sind heute ohne Software nicht mehr funktionsfähig. Damit wächst auch die Verantwortung, die Entwicklerinnen und Entwickler für die Qualität, Sicherheit und Verlässlichkeit ihrer Arbeit tragen.

Technologische Entwicklungszyklen haben sich in den vergangenen Jahren drastisch verkürzt. Was vor zehn Jahren als fortschrittlich galt — etwa containerisierte Deployments oder automatisierte Tests — gehört heute zum Basishandwerk. Large Language Models (LLMs) wie GPT-4 oder GitHub Copilot stehen exemplarisch für einen weiteren Paradigmenwechsel: Erstmals können Teile der eigentlichen Programmierarbeit an KI-Systeme delegiert werden (vgl. (GitHub, Inc., 2023, S. 12)). Dies verschiebt die Wertschöpfung weg von der mechanischen Codeerstellung hin zu konzeptionellen, architekturellen und kommunikativen Tätigkeiten.

1.1 Motivation und Zielsetzung

Bereits im Jahr 2023 identifizierte das World Economic Forum, dass durch Automatisierung und KI bis 2027 rund 83 Millionen Arbeitsplätze wegfallen, gleichzeitig aber 69 Millionen neue entstehen werden — viele davon in der Softwarebranche (vgl. (World Economic Forum, 2023, S. 30)). Daraus ergibt sich die zentrale Frage dieser Arbeit: Welche Kompetenzen werden Softwareentwicklerinnen und -entwickler in Zukunft benötigen, um wettbewerbsfähig, produktiv und professionell zu bleiben?

Ziel dieser Arbeit ist es, die wichtigsten Skill-Kategorien für die Softwarebranche der Zukunft zu identifizieren und zu systematisieren. Dabei werden sowohl technische als auch methodische und überfachliche Kompetenzen betrachtet. Die Erkenntnisse sollen als Orientierungshilfe für Studierende und Berufseinsteiger dienen, die ihre Karriere in der Softwareentwicklung planen. Darüber hinaus können sie als Reflexionsgrundlage für erfahrene Praktikerinnen und Praktiker dienen, die ihre Kompetenzprofile weiterentwickeln möchten.

1.2 Abgrenzung und Aufbau

Die vorliegende Arbeit konzentriert sich auf den Zeithorizont von heute bis etwa 2030. Branchentrends werden auf Basis öffentlich verfügbarer Studien und Berichte analysiert. Eine empirische Primärerhebung — etwa in Form von Experteninterviews oder Unternehmensum-

fragen — ist nicht Gegenstand dieser Arbeit. Die Analyse beschränkt sich bewusst auf die Softwareentwicklung im engeren Sinne und klammert angrenzende Felder wie IT-Management, Produktmanagement oder reine Datenwissenschaft weitgehend aus, wenngleich Überschneidungen unvermeidlich sind.

Die Arbeit gliedert sich wie folgt: Kapitel 2 beleuchtet technische Kernkompetenzen, Kapitel 3 behandelt methodische und prozessuale Skills, Kapitel 4 widmet sich überfachlichen Kompetenzen, und Kapitel 5 fasst die Ergebnisse zusammen, leitet Handlungsempfehlungen ab und gibt einen Ausblick auf zukünftige Entwicklungen.

1.3 Begriffliche Grundlagen

Der Begriff **Skill** wird in dieser Arbeit im Sinne einer erlernbaren, situativ anwendbaren Fähigkeit verwendet. Er umfasst sowohl deklaratives Wissen (Kenntnis von Konzepten und Technologien) als auch prozedurales Wissen (die Fähigkeit zur praktischen Anwendung). In Abgrenzung dazu bezeichnet **Kompetenz** die übergeordnete Befähigung, Wissen, Fertigkeiten und Haltungen in komplexen Situationen zielgerichtet einzusetzen (vgl. (Sommerville, 2015, S. 8 f.)).

Unter **Softwarebranche** werden in dieser Arbeit alle Unternehmen und Organisationen verstanden, die Software professionell entwickeln, betreiben oder vertreiben — unabhängig davon, ob dies ihr Kerngeschäft ist oder als Unterstützungsfunktion erfolgt. Mit der fortschreitenden Digitalisierung ist die Unterscheidung zwischen Softwareunternehmen und traditionellen Branchen zunehmend obsolet: Automobilhersteller, Banken und Energieversorger beschäftigen heute mehr Softwareentwicklerinnen und -entwickler als viele klassische IT-Unternehmen.

2 Technische Kernkompetenzen

Technische Fähigkeiten bilden das Fundament jeder Karriere in der Softwarebranche. Während spezifische Technologien einem schnellen Wandel unterliegen, lassen sich übergeordnete Kompetenzfelder identifizieren, die auch mittelfristig relevant bleiben. Die nachfolgenden Abschnitte analysieren drei Bereiche, die nach aktuellem Stand der Fachdiskussion besondere Relevanz besitzen.

2.1 KI-Kompetenz und Prompt Engineering

Die Fähigkeit, KI-Werkzeuge effektiv einzusetzen, entwickelt sich zur zentralen Schlüsselkompetenz der Softwareentwicklung. LLM-basierte Assistenzsysteme wie GitHub Copilot haben die Produktivität von Entwicklerinnen und Entwicklern in kontrollierten Studien um bis zu 55 % gesteigert, insbesondere bei repetitiven Coding-Aufgaben und beim Schreiben von Boilerplate-Code (vgl. (GitHub, Inc., 2023, S. 14 ff.)). Diese Zahlen sind jedoch mit Vorsicht zu interpretieren: Produktivitätsgewinne bei einfachen Aufgaben können von Qualitätseinbußen bei komplexen Designentscheidungen begleitet sein, wenn Entwicklerinnen und Entwickler den generierten Code unkritisch übernehmen.

Dabei reicht es nicht aus, derartige Werkzeuge passiv zu verwenden. Softwareentwickler müssen verstehen, wie sie mit diesen Systemen kommunizieren, ihre Grenzen und Halluzinationstendenzen kennen und die generierten Ergebnisse kritisch bewerten. **Prompt Engineering** bezeichnet die Fähigkeit, Anfragen an LLMs so zu formulieren, dass präzise und verwertbare Ergebnisse entstehen. Dazu gehören Techniken wie Chain-of-Thought-Prompting, Few-Shot-Beispiele und strukturierte Ausgabeformate. Diese Kompetenz gewinnt in der täglichen Arbeit mit Code-Assistenten, Dokumentationswerkzeugen und NLP-basierten Testsystemen zunehmend an Bedeutung.

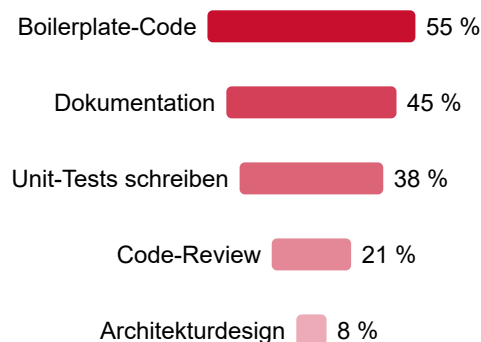


Abbildung 1: Produktivitätssteigerung durch KI-Assistenten nach Aufgabentyp (vgl. (GitHub, Inc., 2023, S. 14 ff.))

Über das reine Prompting hinaus gewinnt auch das Verständnis von KI-Architekturen an Relevanz. Entwicklerinnen und Entwickler, die grob verstehen, wie Transformer-Modelle funk-

tionieren, welche Daten sie verarbeiten und wo ihre systematischen Schwächen liegen, können fundierter entscheiden, wann KI-Unterstützung sinnvoll ist und wann menschliche Expertise unverzichtbar bleibt.

2.2 Cloud-native Entwicklung und DevOps

Modern entwickelte Software wird überwiegend in Cloud-Umgebungen betrieben. Das Stack Overflow Developer Survey 2023 zeigt, dass über 70 % der befragten Entwicklerinnen und Entwickler täglich mit Cloud-Plattformen arbeiten (vgl. (Stack Overflow, 2023)). Die Beherrschung von Container-Technologien (Docker, Kubernetes), Infrastructure-as-Code-Werkzeugen (Terraform, Pulumi) sowie CI/CD-Pipelines gehört damit zum Basiswerkzeug moderner Softwareentwicklung.

DevOps hat sich als kulturelles und technisches Paradigma etabliert, das Entwicklung und Betrieb vereint. Forsgren et al. zeigen in ihrer umfangreichen empirischen Untersuchung, dass High-Performing-Teams durch DevOps-Praktiken eine deutlich höhere Deploymentfrequenz bei gleichzeitig geringerer Change-Failure-Rate erreichen (vgl. (Forsgren, Humble und Kim, 2018, S. 32 ff.)). Die Autoren identifizieren vier Schlüsselmetriken — Deployment Frequency, Lead Time for Changes, Mean Time to Recovery und Change Failure Rate — die seitdem unter dem Begriff DORA Metrics als Industriestandard für die Messung von Software-Delivery-Performance etabliert sind.

Site Reliability Engineering (SRE) als Spezialdisziplin erweitert das DevOps-Konzept um systematische Ansätze zur Zuverlässigkeitsmessung. Konzepte wie Service Level Objectives (SLOs), Error Budgets und Toil Reduction sind Werkzeuge, die ursprünglich von Google entwickelt wurden und inzwischen branchenweite Verbreitung gefunden haben. Kenntnisse in diesem Bereich sind insbesondere für Entwicklerinnen und Entwickler relevant, die in der Verantwortung für produktive Systeme stehen.

2.3 Systemverständnis und Architekturkenntnisse

Trotz zunehmender Abstraktion durch Cloud-Dienste und KI-gestützte Entwicklung bleibt ein solides Verständnis von Systemarchitekturen unverzichtbar. Entwicklerinnen und Entwickler, die verstehen, wie verteilte Systeme, Datenbanken und Netzwerke zusammenwirken, können bessere Entwurfsentscheidungen treffen und Probleme schneller diagnostizieren (vgl. (Somerville, 2015, S. 243 ff.)).

Mikroservice-Architekturen, Event-Driven Design und API-first-Ansätze sind Muster, die in der modernen Softwareentwicklung dominieren. Das Verständnis ihrer Trade-offs — etwa zwischen Konsistenz und Verfügbarkeit im Sinne des CAP-Theorems, oder zwischen Autonomie

einzelner Services und der Komplexität ihrer Orchestrierung — ist ein Merkmal erfahrener Softwarearchitektinnen und -architekten. Hinzu kommt die Kenntnis von Datenbankparadigmen: Relationale Datenbanken, dokumentenbasierte NoSQL-Systeme, Graphdatenbanken und In-Memory-Stores lösen unterschiedliche Probleme und erfordern unterschiedliche Denkmodelle.

Besonders im Kontext von KI-gestützten Anwendungen gewinnt das Verständnis von Datenpipelines und Feature Stores an Bedeutung. MLOps — die Übertragung von DevOps-Prinzipien auf Machine-Learning-Systeme — ist ein aufkommendes Kompetenzfeld, das die Grenze zwischen klassischer Softwareentwicklung und Datenwissenschaft zunehmend verwischt.

3 Methodische und prozessuale Kompetenzen

Neben technischen Fähigkeiten gewinnen methodische Kenntnisse zunehmend an Bedeutung. Die Art und Weise, wie Software-Teams zusammenarbeiten, Prozesse gestalten und Qualität sichern, entscheidet maßgeblich über den Projekterfolg — unabhängig von den eingesetzten Technologien. Die nachfolgenden Abschnitte beleuchten drei Bereiche, die für die professionelle Softwareentwicklung der Zukunft besonders relevant sind.

3.1 Agile Entwicklungsmethoden und kontinuierliches Liefern

Agile Vorgehensmodelle wie Scrum und Kanban haben sich in der Softwarebranche weitgehend durchgesetzt. Sie ermöglichen schnelle Feedbackzyklen, erhöhen die Anpassungsfähigkeit an sich ändernde Anforderungen und fördern die Transparenz im Team (vgl. (Forsgren, Humble und Kim, 2018, S. 18 ff.)). Die empirische Evidenz für ihren Mehrwert ist inzwischen breit: Teams, die kurze Iterationszyklen und regelmäßige Retrospektiven praktizieren, weisen nachweislich höhere Deploymentfrequenzen und niedrigere Fehlerraten auf.

Die Fähigkeit, in iterativen Prozessen zu denken und kontinuierlich lieferbare Software-Inkrementen zu produzieren, ist ein Kernmerkmal moderner Softwareentwicklung. Dabei reicht das Wissen über agile Methoden allein nicht aus — entscheidend ist das Verständnis der zugrunde liegenden Werte: Individuen und Interaktionen über Prozesse und Werkzeuge, funktionierende Software über umfassende Dokumentation, Zusammenarbeit mit dem Kunden über Vertragsverhandlungen, Reaktion auf Veränderung über das Befolgen eines Plans. Diese Prinzipien des Agile Manifesto bilden den kulturellen Rahmen, ohne den agile Methoden zu bloßen Prozessritualen degenerieren.

Continuous Delivery — die Fähigkeit, Software jederzeit in einem auslieferbaren Zustand zu halten — ist die technische Entsprechung agiler Entwicklungskultur. Sie erfordert nicht nur technische Automatisierung, sondern auch eine organisationale Bereitschaft, kleine, häufige Änderungen gegenüber großen, seltenen Releases zu bevorzugen. Diese Denkweise steht im Widerspruch zu klassischen Projektmanagementansätzen und erfordert einen bewussten kulturellen Wandel.



Abbildung 2: Schematische Darstellung einer CI/CD-Pipeline (eigene Darstellung)

3.2 Qualitätssicherung und testgetriebene Entwicklung

Softwarequalität ist kein nachgelagerter Schritt, sondern muss von Beginn an in den Entwicklungsprozess integriert werden. Test-Driven Development (TDD) und Behavior-Driven Development (BDD) sind etablierte Ansätze, um Qualität zu sichern.

pment (BDD) sind Ansätze, die Qualität durch gezielte Testautomatisierung sicherstellen. Bei TDD wird der Test vor dem produktiven Code geschrieben, was zu einer höheren Testabdeckung, besserem Design und weniger Regressionen führt (vgl. (Sommerville, 2015, S. 207 ff.)).

Automatisierte Testsuites auf Unit-, Integrations- und End-to-End-Ebene ermöglichen es, Regressionen frühzeitig zu erkennen und Deployments mit hohem Vertrauen durchzuführen (vgl. (Forsgren, Humble und Kim, 2018, S. 55)). Diese Fähigkeit wird durch den zunehmenden Einsatz von CI/CD-Pipelines noch wichtiger, da manuelle Qualitätssicherung bei hohen Deploymentfrequenzen nicht mehr praktikabel ist. Code Reviews — die systematische gegenseitige Überprüfung von Code durch Teammitglieder — ergänzen automatisierte Tests um eine qualitative, menschliche Perspektive und fördern gleichzeitig den Wissenstransfer im Team.

Statische Codeanalyse, Linting und automatisierte Sicherheitsscans sind weitere Werkzeuge, die in modernen Entwicklungspipelines selbstverständlich geworden sind. Die Fähigkeit, diese Werkzeuge sinnvoll zu konfigurieren, ihre Ausgaben zu interpretieren und false positives von echten Problemen zu unterscheiden, gehört zum Handwerkszeug professioneller Entwicklerinnen und Entwickler.

3.3 Sicherheitskompetenz (Security by Design)

Mit der steigenden Vernetzung von Systemen wächst die Angriffsfläche für Cyberangriffe erheblich. Datenschutzverletzungen, Ransomware-Angriffe und Lieferkettenangriffe auf Software-Abhängigkeiten haben in den vergangenen Jahren massiv zugenommen und verursacht weltweit Schäden in Milliardenhöhe. Der Ansatz **Security by Design** fordert, dass Sicherheitsaspekte nicht nachträglich ergänzt, sondern von Beginn an in die Softwarearchitektur integriert werden.

Entwicklerinnen und Entwickler sollten mit grundlegenden Sicherheitskonzepten wie Authentifizierung, Autorisierung, Verschlüsselung und dem Prinzip der minimalen Rechtevergabe (Principle of Least Privilege) vertraut sein. Das Wissen um häufige Angriffsvektoren — SQL-Injection, Cross-Site Scripting, Insecure Deserialization oder Server-Side Request Forgery — ist ebenso unverzichtbar wie der sichere Umgang mit Secrets und Zugangsdaten in automatisierten Pipelines. Mit der Verbreitung von Softwarelieferketten (Supply Chains) und dem Einsatz externer Abhängigkeiten über Paketmanager rücken auch Dependency-Schwachstellen und Software Bill of Materials (SBOM) in den Fokus professioneller Qualitätssicherung.

Die OWASP Top 10 — eine jährlich aktualisierte Liste der kritischsten Webanwendungsschwachstellen — bietet eine praxisnahe Orientierung für Entwicklerinnen und Entwickler, die sicherheitsrelevante Grundkenntnisse systematisch aufbauen möchten. Ergänzt durch Threat Modeling und regelmäßige Penetrationstests bildet Security-Kompetenz einen unverzichtbaren Bestandteil professioneller Softwareentwicklung.

4 Überfachliche Kompetenzen

Technisches Wissen allein reicht nicht aus, um in der modernen Softwarebranche erfolgreich zu sein. Überfachliche Kompetenzen — häufig als Soft Skills bezeichnet, obwohl dieser Begriff ihre Bedeutung unzureichend widerspiegelt — sind entscheidend für Zusammenarbeit, Kommunikation und kontinuierliches Wachstum. In einer Branche, in der technische Grundfähigkeiten durch KI-Assistenten zunehmend zugänglich werden, werden gerade diese Kompetenzen zum differenzierenden Faktor.

4.1 Kommunikation und Kollaboration

Frederick Brooks formulierte bereits 1975 in seinem wegweisenden Werk **The Mythical Man-Month**, dass die größten Herausforderungen in der Softwareentwicklung nicht technischer, sondern sozialer und kommunikativer Natur sind (vgl. (Brooks, 1995, S. 17 f.)). Diese Einschätzung hat bis heute Bestand — und gewinnt in einer Zeit verteilter, internationaler Teams zusätzlich an Gewicht.

Softwareentwicklerinnen und -entwickler arbeiten in crossfunktionalen, häufig global verteilten Teams. Die Fähigkeit, technische Sachverhalte klar und verständlich zu kommunizieren — gegenüber Kolleginnen und Kollegen ebenso wie gegenüber fachfremden Stakeholdern — ist ein wesentlicher Erfolgsfaktor. Dazu gehören die schriftliche Kommunikation (technische Dokumentation, Pull-Request-Beschreibungen, Architecture Decision Records) ebenso wie mündliche Präsentation in Sprint Reviews, Architektur-Diskussionen und Kundenterminen.

Eng verknüpft mit Kommunikation ist die Fähigkeit zur konstruktiven Zusammenarbeit: Code Reviews geben und annehmen, Feedback konstruktiv formulieren, in multidisziplinären Teams auf ein gemeinsames Ziel hinarbeiten und Konflikte sachlich austragen. Diese Fähigkeiten lassen sich erlernen und trainieren — sie entstehen jedoch nicht automatisch durch den Einsatz agiler Methoden, sondern erfordern bewusste Reflexion und Übung. Remote-Arbeit, die sich nach der COVID-19-Pandemie dauerhaft etabliert hat, stellt zusätzliche Anforderungen an asynchrone Kommunikation und schriftliche Ausdrucksfähigkeit.

4.2 Lernbereitschaft und Adaptionfähigkeit

Keine andere Branche verändert sich schneller als die Softwarebranche. Programmiersprachen, Frameworks und Paradigmen kommen und gehen; wer heute ausschließlich auf eine bestimmte Technologie setzt, muss morgen möglicherweise fundamental Neues erlernen. Das Stack Overflow Developer Survey 2023 zeigt, dass ein erheblicher Anteil der Entwicklerinnen und Entwickler jährlich neue Programmiersprachen oder Frameworks erlernt (vgl. (Stack

Overflow, 2023)). Entscheidend ist daher nicht das Beherrschen einer bestimmten Technologie, sondern die Fähigkeit, schnell und gezielt Neues zu lernen.

Carol Dweck prägte den Begriff des **Growth Mindset** — die Überzeugung, dass Fähigkeiten durch Anstrengung und Lernen entwickelt werden können, anstatt als unveränderlich zu gelten. In der Softwareentwicklung manifestiert sich dieses Mindset in der aktiven Auseinandersetzung mit neuen Entwicklungen: Lektüre von Fachartikeln und technischen Blogs, Beitrag zu Open-Source-Projekten, Teilnahme an Konferenzen und Meetups sowie die Nutzung von Online-Lernplattformen. Kontinuierliches Lernen ist damit kein gelegentliches Weiterbildungsprojekt, sondern ein integraler Bestandteil professioneller Softwareentwicklung.

Die Adaptionfähigkeit betrifft auch den Umgang mit Unsicherheit und Ambiguität. Anforderungen ändern sich, Technologien werden obsolet, Architekturen müssen revidiert werden. Entwicklerinnen und Entwickler, die mit dieser Ungewissheit produktiv umgehen können, anstatt auf vollständige Spezifikationen zu warten, sind in einer agilen Umgebung besonders wertvoll.



Abbildung 3: T-förmiges Kompetenzprofil als Karrieremodell (eigene Darstellung)

4.3 Kritisches Denken und Problemlösekompetenz

Komplexe Softwareprobleme lassen sich selten durch bloßes Anwenden erlernter Muster lösen. Gefragt ist analytisches, strukturiertes Denken: das Zerlegen eines Problems in lösbare Teilprobleme, das Aufstellen und Testen von Hypothesen, das Evaluieren von Trade-offs zwischen konkurrierenden Lösungsansätzen und das Abschätzen langfristiger Konsequenzen von Architekturentscheidungen.

Besonders im Zeitalter KI-generierter Codevorschläge wird kritisches Denken wichtiger denn je. Entwicklerinnen und Entwickler müssen in der Lage sein, automatisch generierten Code zu hinterfragen, Sicherheitslücken zu erkennen und die semantische Korrektheit von Lösungen zu beurteilen — auch wenn diese syntaktisch fehlerfrei sind. Die Fähigkeit, das **Warum** hinter einer

Lösung zu verstehen und nicht nur das **Was**, bleibt eine genuinen menschlichen Kompetenz, die durch KI bisher nicht ersetzt wird.

Problemlösekompetenz umfasst darüber hinaus die Fähigkeit zur Priorisierung: In einem System mit begrenzten Ressourcen und konkurrierenden Anforderungen müssen ständig Entscheidungen getroffen werden — welcher Bug wird zuerst behoben, welche technische Schuld wird wann abgebaut, welches Feature hat den höchsten Mehrwert für die Nutzerinnen und Nutzer. Diese Fähigkeit verbindet technisches mit strategischem Denken und ist eine wesentliche Voraussetzung für die Übernahme von Verantwortung in wachsenden Rollen.

5 Fazit und Ausblick

Die vorangegangene Analyse zeigt, dass die Softwarebranche der Zukunft eine breite Palette an Kompetenzen erfordert, die weit über klassisches Programmieren hinausgehen. Technische Exzellenz bleibt notwendig, ist aber allein nicht mehr hinreichend. Die identifizierten Skill-Kategorien lassen sich in einem Kompetenzmodell auf drei Ebenen zusammenfassen:

Ebene	Kompetenzbereich	Beispiele
Technisch	KI & LLMs, Cloud-native, Architektur	Prompt Engineering, Kubernetes, Microservices, MLOps
Methodisch	Agile, Qualitätssicherung, Security	Scrum, TDD, CI/CD, Security by Design, OWASP
Überfachlich	Kommunikation, Lernen, kritisches Denken	Code Reviews, Growth Mindset, Problemanalyse

Tabelle 1: Kompetenzmodell für die Softwarebranche der Zukunft (eigene Darstellung)

5.1 Zusammenfassung der Ergebnisse

Drei übergreifende Trends prägen die Kompetenzanforderungen der kommenden Jahre:

Erstens verschiebt sich die Rolle der Entwicklerin bzw. des Entwicklers vom reinen Coder zum Orchestrator: KI-Werkzeuge übernehmen repetitive Codieraufgaben, während Menschen zunehmend für Architekturentscheidungen, Qualitätssicherung und strategische Weichenstellungen verantwortlich sind (vgl. (World Economic Forum, 2023, S. 33)). Diese Verschiebung erhöht die Anforderungen an konzeptionelles Denken und reduziert gleichzeitig die Einstiegshürde für syntaktisch korrekte Codeerzeugung. Die Konsequenz ist eine Spreizung der Anforderungsprofile: Einfache Implementierungsaufgaben werden zunehmend automatisiert, während die Nachfrage nach erfahrenen Architektinnen und Architekten, Senior-Entwicklerinnen und -Entwicklern sowie technischen Führungspersonlichkeiten weiter steigt.

Zweitens verschwimmen die Grenzen zwischen Entwicklung, Betrieb und Sicherheit. Full-Stack-Kompetenz, die Cloud, DevOps und Security gleichermaßen umfasst, wird zur Basiserwartung an erfahrene Fachkräfte (vgl. (Forsgren, Humble und Kim, 2018, S. 11 ff.)). Die klassische Trennung zwischen Entwicklung und Betrieb — exemplarisch durch den Satz „it works on my machine“, symbolisiert — ist in modernen Organisationen weitgehend überwunden. Gleichzeitig steigen die regulatorischen Anforderungen an Datenschutz und IT-Sicherheit, etwa durch den European Cyber Resilience Act oder die NIS2-Richtlinie, was Security-Kompetenz von einer optionalen Spezialisierung zur Pflichtanforderung macht.

Drittens steigt die Bedeutung überfachlicher Kompetenzen relativ zu rein technischen Fähigkeiten. In einer Welt, in der technisches Grundwissen durch KI-Assistenten zunehmend zugänglich wird, differenzieren Kommunikationsfähigkeit, Urteilsvermögen und Adaptionsefähigkeit (vgl. (Brooks, 1995, S. 42 f.)). Diese Kompetenzen sind schwerer automatisierbar und damit langfristig wertbeständiger als spezifisches Technologiewissen.

5.2 Handlungsempfehlungen

Für Studierende und Berufseinsteiger in der Softwarebranche ergeben sich aus den vorangegangenen Analysen folgende konkrete Empfehlungen:

1. **KI-Werkzeuge aktiv nutzen und kritisch hinterfragen:** KI-Assistenten sollten in den Arbeitsalltag integriert werden, um Produktivitätspotenziale zu erschließen. Gleichzeitig ist die Fähigkeit zur kritischen Evaluation generierten Codes unverzichtbar — sowohl in Bezug auf Korrektheit als auch auf Sicherheit und Wartbarkeit.
2. **T-förmiges Kompetenzprofil aufbauen:** Eine breite Grundlage kombiniert mit tiefer Expertise in einem oder zwei Bereichen bleibt ein tragfähiges Karrieremodell. Die Wahl des Tiefenbereichs sollte persönliche Interessen mit Marktrelevanz verbinden.
3. **Lernen als permanenten Prozess verstehen:** Regelmäßige Lernzeiten sollten fest in den Arbeitsalltag integriert werden. Open-Source-Beiträge, technische Blogs, Konferenzen und Community-Engagement sind effektive Wege, um am Puls der Entwicklung zu bleiben.
4. **Überfachliche Kompetenzen gezielt entwickeln:** Möglichkeiten zur Präsentation, zum Moderieren von Meetings und zur cross-funktionalen Zusammenarbeit sollten aktiv gesucht werden. Schreibkompetenz — für Dokumentation, Pull Requests und technische Konzepte — verdient besondere Aufmerksamkeit.
5. **Sicherheit als Querschnittsthema verankern:** Sicherheitskonzepte sollten nicht als separates Fachgebiet, sondern als integraler Bestandteil jeder Entwicklungstätigkeit betrachtet werden. Die OWASP Top 10 und grundlegende Kryptographiekennntnisse bilden eine sinnvolle Ausgangsbasis.

5.3 Ausblick

Die Softwarebranche wird sich in den kommenden Jahren weiter dynamisch entwickeln. Themen wie Quantum Computing, neuromorphe Hardware und autonome Agentensysteme werden neue Spezialisierungsfelder eröffnen und bestehende Paradigmen herausfordern. Gleichzeitig ist davon auszugehen, dass die in dieser Arbeit beschriebenen Grundkompetenzen — kritisches Denken, Lernbereitschaft, Systemverständnis, Kommunikationsfähigkeit —

ihre Relevanz behalten werden. Sie sind nicht technologiegebunden, sondern beschreiben eine professionelle Haltung, die in einer sich wandelnden Branche langfristig trägt.

Offen bleibt die Frage, wie sich das Berufsbild der Softwareentwicklerin bzw. des Softwareentwicklers mittelfristig verändern wird, wenn KI-Systeme zunehmend eigenständig Code generieren, testen und deployen können. Es ist wahrscheinlich, dass sich der Beruf weiter in Richtung technischer Führung, Systemverantwortung und interdisziplinärer Gestaltung verschiebt — weg von der Implementierung und hin zur Orchestrierung. Diese Entwicklung stellt hohe Anforderungen an die Ausbildung und die kontinuierliche Weiterbildung von Fachkräften und ist eine zentrale Aufgabe für Hochschulen, Unternehmen und die Branche insgesamt.

Anhang

A1: Übersicht eingesetzter KI-Werkzeuge

Werkzeug	Beschreibung der Nutzung
GitHub Copilot	• Autovervollständigung von Code-Fragmenten beim Schreiben der Kapitelbeispiele (gesamt) • Generierung von Typst-Tabellengerüsten für das Kompetenzmodell (Kapitel 5)
ChatGPT (GPT-4o)	• Recherche und Strukturierung von Fachinhalten zu KI-Kompetenz und Prompt Engineering (Kapitel 2.1) • Verständnis von DevOps-Konzepten und DORA Metrics (Kapitel 2.2) • Identifikation relevanter Literaturstellen und Prüfung der Quellenlage (gesamt) • Formulierungshilfe und Stiloptimierung einzelner Textpassagen (gesamt)
Microsoft Copilot (Office 365)	• Korrektur- und Formulierungshilfe für den deutschen Fließtext (gesamt) • Zusammenfassung und Auswertung des WEF Future of Jobs Report 2023 (Kapitel 1 und 5)
Claude (Anthropic)	• Unterstützung bei der Erstellung der Typst-Vorlage und Formatierungsfragen (gesamt) • Überprüfung der inhaltlichen Konsistenz zwischen Kapiteln (gesamt)

Tabelle 2: Übersicht über die Nutzung von KI-basierten Werkzeugen

Literaturverzeichnis

Brooks, F.P. (1995) *The Mythical Man-Month: Essays on Software Engineering*. 2. Aufl. Reading, MA: Addison-Wesley.

Forsgren, N., Humble, J. und Kim, G. (2018) „Accelerate: The Science of Lean Software and DevOps“, *IT Revolution Press* [Preprint].

GitHub, Inc. (2023) „The State of the Octoverse 2023“, *GitHub Blog* [Preprint].

Sommerville, I. (2015) *Software Engineering*. 10. Aufl. Harlow: Pearson.

Stack Overflow (2023) „Developer Survey 2023“.

World Economic Forum (2023) *Future of Jobs Report 2023*. Geneva: World Economic Forum.

Selbständigkeitserklärung

Ich versichere hiermit, dass ich die vorliegende Seminararbeit mit dem Thema

Skills der Zukunft in der Softwarebranche

Kompetenzen und Qualifikationen im Zeitalter von KI und digitaler Transformation

selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt habe und diese bislang nicht veröffentlicht wurde. Des Weiteren versichere ich, dass die eingereichte elektronische Fassung mit der gedruckten Ausfertigung übereinstimmt.

Ort, Datum

Unterschrift

Erklärung und Nutzungsdokumentation zum Einsatz von KI-basierten Werkzeugen bei der Anfertigung von wissenschaftlichen Arbeiten als Prüfungsleistungen

Zur Verwendung KI-gestützter Werkzeuge erkläre ich in Kenntnis des Hinweisblatts „Hinweise zum Einsatz von KI-basierten Werkzeugen bei der Anfertigung von wissenschaftlichen Arbeiten als Prüfungsleistungen im prüfungsrechtlichen Kontext“ Folgendes:

- Ich habe mich aktiv über die Leistungsfähigkeit und Beschränkungen der in meiner Arbeit eingesetzten KI-Werkzeuge informiert.
- Bei der Anfertigung der Arbeit habe ich durchgehend eigenständig und beim Einsatz KI-gestützter Werkzeuge maßgeblich steuernd gearbeitet.
- Insbesondere habe ich die Inhalte entweder aus wissenschaftlicher oder anderen zugelassenen Quellen entnommen und diese gekennzeichnet oder diese unter Anwendung wissenschaftlicher Methoden selbst entwickelt.
- Mir ist bewusst, dass ich als Verfasser:in der Arbeit die volle Verantwortung für die in ihr gemachten Angaben und Aussagen trage.
- Soweit ich KI-gestützte Werkzeuge zur Erstellung der Arbeit eingesetzt habe, sind diese zu den Produktnamen, den formulierten Eingaben (Prompts), den Einsatzzwecken und der entsprechenden Seiten-/Bereichsreferenzierung auf die Arbeit im KI-Verzeichnis am Ende der Arbeit vollständig ausgewiesen und im Text belegt (z.B. als Fußnote).

Ort, Datum

Unterschrift